DOCUMENTATION TECHNIQUE - POLE INFRA

Introduction du pôle INFRA 1
Présentation des besoins 1
Solution mise en place 2
Création du double réseau WiFi 2
Mise en place du tunnel WireGuard 3
Sécurisation avec UFW et Fail2Ban 5
Points de troubleshooting rencontrés 7
Installation de Docker sur le VPS 7
Mise en place de Grafana et Prometheus 7
Mise en place de PostgreSQL en Docker Compose 14
Mise en place de Redis sur le VPS 15
Installer le tunnel VPN entre les deux VPS 17
Supervision et gestion future 18
Proposition Cloud 19
Conclusion 21

Introduction du pôle INFRA

Le pôle infrastructure a pour but de mettre en place et maintenir l'ensemble des ressources nécessaires au bon fonctionnement global du projet. C'est notamment lui qui va lier l'ensemble des éléments fournis par les autres équipes pour les mettre à disposition du grand public de façon sécurisée et pleinement fonctionnelle.

Présentation des besoins

Pour ce projet les besoins liés à l'infrastructure étaient séparés en 3 pôles :

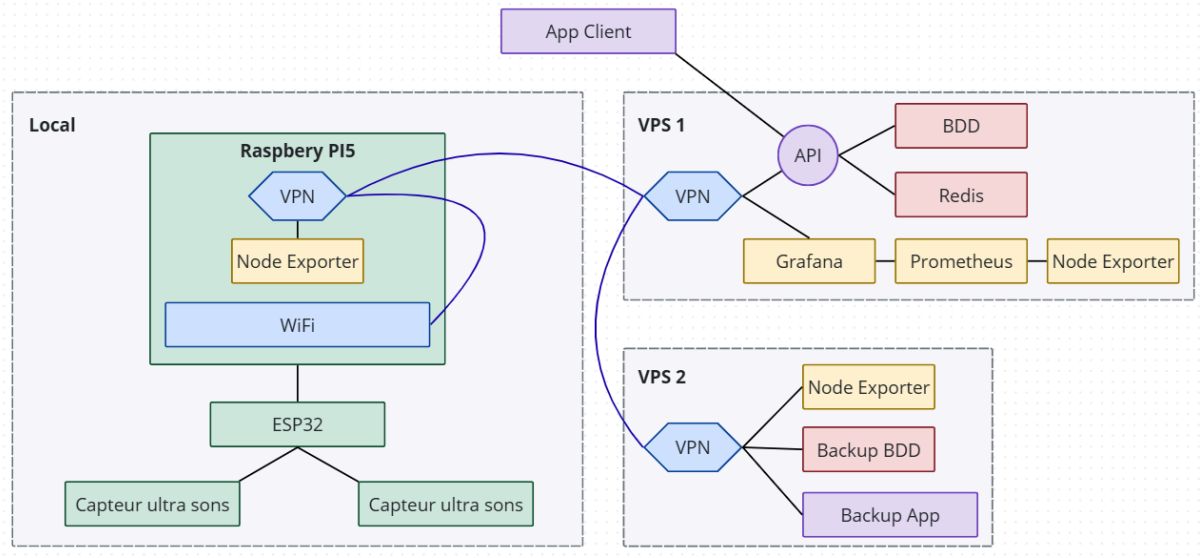
- Déployer la partie applicative : API et base de données
- Mettre en place le réseau global : WiFi IOT et liaison inter-serveurs sécurisée
- Surveiller et maintenir l'infra : supervision et monitoring des métriques

Ainsi l'objectif était de permettre la mise en place du projet n'importe où et n'importe quand, une simple connexion internet sera nécessaire (un partage de connexion 4G peut même être utilisé) afin de lier le système électronique avec l'ensemble des serveurs pour faire remonter les données sur l'application. De plus un enjeux de

sécurité était présent, il fallait faire communiquer plusieurs serveurs sans pour autant prendre le risque d'exposer inutilement des ports. Enfin le monitoring et la supervision étaient une part importante de l'infra, l'objectif étant d'asauvoir un point de vue centralisé pour permettre de surveiller l'ensemble du système et de faire remonter les problèmes rapidement.

Solution mise en place

Schéma de l'infrastructure déployée :



Création du double réseau WiFi

Pour permettre au Raspberry d'avoir accès à internet et d'émettre le hotspot pour l'IOT nous allons devoir mettre en place deux réseaux WiFi distincts avec deux cartes réseaux différentes. Nous allons donc réserver la carte interne pour le Hotspot "Ynough" qui permettra de connecter les ESP32 et les appareils du staff ainsi que l'accès WAN pour permettre au Raspberry de communiquer avec le reste de l'infrastructure (cet accès peut être un WiFi public, un partage de connexion 4G ou bien un câble RJ45).

Pour commencer nous allons connecter le Raspberry à internet via une carte réseau USB supplémentaire (ici ce sera le réseau IOT de l'école) et attribuer cette configuration à l'adresse mac de la carte réseau (pour permettre une persistance au redémarrage) :

```
sudo nmcli device wifi connect "IOT@YNOV" password '<mot-de-passe-wifi>'
ifname wlan0

sudo nmcli connection modify "IOT@YNOV" 802-11-wireless.mac-address
"18:A6:F7:16:F4:AC" connection.interface-name "" connection.autoconnect
yes
```

Ensuite nous allons créer le Hotspot local :

```
sudo nmcli device wifi hotspot ifname wlan1 ssid "Ynough"
password '<mot-de-passe-hotspot>'

sudo nmcli connection modify "Hotspot"
802-11-wireless.mac-address "88:A2:9E:9D:A9:57"
connection.interface-name ""

sudo nmcli connection modify "Hotspot" ipv4.method shared

sudo nmcli connection modify "Hotspot" connection.autoconnect
yes
```

Toute la partie DHCP est nativement gérée par le Raspberry via dnsmasq qui fournira des IP aux clients du Hotspot sur le réseau 10.42.0.0

Mise en place du tunnel WireGuard

Avant l'installation, le plan réseau retenu initialement (avant l'installation du VPS2) était le suivant :

- VPS Debian : IP publique 94.125.160.250
- Raspberry Pi : IP publique non définie (partage de connexion)
- Plan d'adressage VPN WireGuard :
  - VPS : 10.99.0.1/24
  - Raspberry Pi : 10.99.0.2/24

WireGuard a été retenu car il s'agit d'un VPN couche 3 léger, rapide et relativement simple à configurer. Le principe repose sur une paire de clés par machine, chaque pair connaissant la clé publique de l'autre et l'adresse privée utilisée dans le tunnel.

Sur le VPS Debian, l'installation et la configuration ont été réalisées avec les commandes suivantes :

```
sudo apt update
sudo apt install -y wireguard

sudo install -m 700 -d /etc/wireguard
cd /etc/wireguard

umask 077
wg genkey | sudo tee privatekey | wg pubkey | sudo tee publickey
```

```
sudo cat /etc/wireguard/privatekey
sudo cat /etc/wireguard/publickey
```

Le fichier de configuration /etc/wireguard/wg0.conf a ensuite été préparé comme suit:

```
[Interface]
Address = 10.99.0.1/24
ListenPort = 51820
PrivateKey = CLE_PRIVEE_VPS

[Peer]
PublicKey = CLE_PUBLIQUE_PI
AllowedIPs = 10.99.0.2/32
```

Dans cette configuration :
- CLE_PRIVEE_VPS correspond au contenu de /etc/wireguard/privatekey
- CLE_PUBLIQUE_PI correspond à la clé publique du Raspberry Pi

Le service a ensuite été activé et démarré :

```
sudo systemctl enable --now wg-quick@wg0
sudo systemctl status wg-quick@wg0
sudo wg show
```

Il faut noter que le port UDP 51820 a également été autorisé plus tard côté pare-feu pour permettre l'établissement du tunnel.

Sur le Raspberry Pi, les opérations ont été similaires :

```
sudo apt update
sudo apt install -y wireguard

sudo install -m 700 -d /etc/wireguard
cd /etc/wireguard

umask 077
wg genkey | sudo tee privatekey | wg pubkey | sudo tee publickey

sudo cat /etc/wireguard/privatekey
sudo cat /etc/wireguard/publickey
```

Le fichier /etc/wireguard/wg0.conf a été configuré ainsi :

```
[Interface]
Address = 10.99.0.2/24
PrivateKey = CLE_PRIVEE_PI
```

```
[Peer]
PublicKey = CLE_PUBLIQUE_VPS
Endpoint = 94.125.160.250:51820
AllowedIPs = 10.99.0.1/32
PersistentKeepalive = 25
```

Dans cette configuration :
- CLE_PRIVEE_PI correspond à la clé privée du Raspberry Pi
- CLE_PUBLIQUE_VPS correspond à la clé publique du VPS

Le service a ensuite été activé et vérifié :

```
sudo systemctl enable --now wg-quick@wg0
sudo systemctl status wg-quick@wg0
sudo wg show
```

Sécurisation avec UFW et Fail2Ban

Pour limiter la surface d'exposition du VPS et du Raspberry Pi, un pare-feu local UFW a été mis en place sur les deux machines. Le principe retenu a été de bloquer tout ce qui entre par défaut et de n'autoriser que les flux nécessaires, en particulier SSH et WireGuard.

Sur le VPS Debian, la configuration UFW a été faite ainsi :

```
sudo apt update
sudo apt install -y ufw

sudo ufw default deny incoming
sudo ufw default allow outgoing

sudo ufw allow 22/tcp
sudo ufw allow 51820/udp

sudo ufw enable
sudo ufw status verbose
```

Le résultat attendu dans l'état du pare-feu était :
- deny en entrée par défaut
- allow en sortie par défaut
- 22/tcp autorisé
- 51820/udp autorisé

La même logique a été appliquée sur le Raspberry Pi mais en incluant des vérifications supplémentaires, notamment que l'ip de provenance était bien celle du VPS ou du réseau WiFi interne :

```
sudo apt update
```

```
sudo apt install -y ufw

sudo ufw default deny incoming
sudo ufw default allow outgoing

sudo ufw allow from 94.125.160.250 to any port 51820 proto udp
sudo ufw allow from 10.42.0.0/24 to any port 22 proto tcp
sudo ufw allow in on wlan1 to any port 67 proto udp
sudo ufw allow in on wlan1 to any port 53
sudo ufw allow in on wg0 from 10.99.0.1 to any port 9100 proto tcp
sudo ufw allow from 192.168.100.0/24 to any port 22 proto tcp
sudo ufw route allow in on wlan1 out on wlan0

sudo ufw enable
sudo ufw status verbose
```

Fail2Ban a été installé sur le VPS pour protéger le service SSH contre les tentatives répétées de connexion échouée. L'objectif était ici de bannir temporairement les IP effectuant trop d'essais ratés.

Les commandes utilisées ont été :

```
sudo apt update
sudo apt install -y fail2ban

sudo systemctl enable fail2ban
sudo nano /etc/fail2ban/jail.local
```

Contenu du fichier /etc/fail2ban/jail.local :

```
[sshd]
port    = 22
logpath = %(sshd_log)s
backend = %(sshd_backend)s
enabled = true
filter = sshd
maxretry = 3
findtime = 5m
bantime  = 30m
```

Après création du fichier, le service a été redémarré et contrôlé :

```
sudo systemctl restart fail2ban
sudo systemctl status fail2ban
sudo fail2ban-client status
sudo fail2ban-client status sshd
```

Points de troubleshooting rencontrés

Au cours de la mise en place, certains problèmes ont été identifiés.

Un premier point concernait l'ouverture de ports nécessaires à certains services, notamment le DHCP. Ces ports n'ayant pas été autorisés au départ, de nouvelles machines n'arrivaient pas à obtenir d'adresse IP correctement. Le problème a ensuite été corrigé par l'ajout de règles supplémentaires:

Un second point concernait un durcissement plus avancé de la sécurité réseau, avec restriction de certaines ouvertures de ports à des sources précises seulement. Par exemple, le port 51820 pouvait être limité à l'adresse IP publique de la machine peer

Installation de Docker sur le VPS

Pour héberger les services nécessaires au projet, Docker a été installé sur le VPS. L'installation a été lancée à l'aide du script suivant :

https://github.com/Entys/Docker-Install_Solution

Mise en place de Grafana et Prometheus

Afin de disposer de monitoring, un stack Docker Compose regroupant Prometheus et Grafana a été déployé sur le VPS.

Prometheus a été choisi pour la collecte de métriques, et Grafana pour la visualisation de ces métriques dans des tableaux de bord.

L'arborescence nécessaire a été créée avec :

```
sudo mkdir -p
/opt/docker/monitoring/{prometheus/data,grafana/data,grafana/provisionin
g/datasources}
cd /opt/docker/monitoring
```

Le fichier .env de Grafana a ensuite été créé :

```
sudo nano /opt/docker/monitoring/.env
```

Contenu :

```
GF_SECURITY_ADMIN_USER=admin
GF_SECURITY_ADMIN_PASSWORD=mot-de-passe-évidement-changé
```

Puis sécurisé avec :

```
sudo chmod 600 /opt/docker/monitoring/.env
```

Le fichier de configuration Prometheus a été créé ici :

```
sudo nano /opt/docker/monitoring/prometheus/prometheus.yml
```

Contenu :

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  - job_name: 'vps-main'
    static_configs:
      - targets: ['10.99.0.1:9100']
        labels:
          instance: 'vps-main'

  - job_name: 'raspberry-pi'
    static_configs:
      - targets: ['10.99.0.2:9100']
        labels:
          instance: 'raspberry-pi'
```

Le provisioning de la datasource côté Grafana a été configuré ici :

```
sudo nano
/opt/docker/monitoring/grafana/provisioning/datasources/prometheus.yml
```

Contenu :

```
apiVersion: 1

datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: true
```

Le fichier Docker Compose a ensuite été créé :

```
sudo nano /opt/docker/monitoring/compose.yaml
```

Contenu :

```yaml
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--storage.tsdb.retention.time=30d'
      -
'--web.console.libraries=/usr/share/prometheus/console_libraries'
      - '--web.console.templates=/usr/share/prometheus/consoles'
    networks:
      - monitoring
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    env_file:
      - .env
    volumes:
      - ./grafana/data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    environment:
      GF_INSTALL_PLUGINS: ""
    ports:
      - '3000:3000'
    networks:
      - monitoring
    restart: unless-stopped
    depends_on:
      - prometheus

networks:
  monitoring:
    driver: bridge
```

Les permissions ont ensuite été ajustées :

```
sudo chown -R 472:472 /opt/docker/monitoring/grafana/data
sudo chmod -R 755 /opt/docker/monitoring/grafana
```

```
sudo chmod -R 755 /opt/docker/monitoring/prometheus

sudo chown -R nobody:nogroup ./prometheus/data
sudo chmod -R 775 ./prometheus/data
```

Enfin, la stack a été lancée et vérifiée :

```
cd /opt/docker/monitoring
sudo docker compose pull
sudo docker compose up -d
sudo docker compose ps
sudo docker compose logs --tail=100
```

Installation node exporter sur le RP5:

- Dans un premier temps créer un utilisateur dédié nommé node_exporteur avec le moins de droits possible pour une meilleure sécurité (pas de shell valide ni de répertoir personnel pour le nouvel utilisateur).

```
sudo useradd -M -r -s /bin/false node_exporter
```

- Dans un second temps il faut procéder à l'installation du binaire (Architecture ARM64) car le RP5 n'est pas compatible par défaut avec node exporter sans le bon code machine.

```
cd /tmp
wget
https://github.com/prometheus/node_exporter/releases/download/v1.8.2/nod
e_exporter-1.8.2.linux-arm64.tar.gz
```

- Ensuite on extrait l'archive, on place l'exécutable dans le répertoire standard et on lui attribue les bons droits pour l'exécution.

```
tar xvfz node_exporter-1.8.2.linux-arm64.tar.gz
sudo mv node_exporter-1.8.2.linux-arm64/node_exporter /usr/local/bin/
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
```

- On nettoie les fichiers temporaires car des fichiers de script peuvent être mal exécutés ou utilisés à des fins malveillants.

```
rm -rf node_exporter-1.8.2.linux-arm64*
```

Maintenant place à la configuration de node exporter:

- Ce service tourne par défaut sur toutes les interfaces réseaux le but va être de restreindre la connexion à l'interface du VPN. Pour bien débuter il faut donc identifier l'IP (mis en place sur wg0 avec wireguard).

```
ip a
```

- Une fois l'ip VPN identifier il faut créer le fichier de configuration node exporter dans le chemin suivant.

```
sudo nano /etc/systemd/system/node_exporter.service
```

- Copier dans le fichier cette configuration en y modifiant (`--web.listen-address`) avec l'IP VPN configuré.

```
[Unit]
Description=Prometheus Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter \
    --web.listen-address=10.99.0.2:9100
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Activation du service node exporter:

- On recharge le daemon du systemd pour qu'il prenne en compte la nouvelle config, on fait en sorte que le service démarre au démarrage du Raspberry et on démarre le service.

```
sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
```

- Important: Voir si le service a bien démarré.

```
sudo systemctl status node_exporter
```

Tester ensuite node exporter:

- Vérifier que le port :9100 est bien ouvert et écoute sur la bonne IP

```
curl http://10.99.0.2:9100/metrics
```

- Pour un test complet se connecter ensuite sur le VPS et tester si le tunnel monitoring marche aussi bien.

```
curl http://10.99.0.2:9100/metrics
```

- Attention à la configuration pare-feu qui peut bloquer le port configurer pour une meilleur pratique fait l'autorisation que pour l'interface wg0 VPN.

```
sudo ufw allow in on wg0 from 10.99.0.1 to any port 9100 proto tcp
```

Installation node exporter sur le VPS:

- Pour commencer créer un utilisateur avec un minimum de privilèges.

```
sudo useradd -M -r -s /bin/false node_exporter
```

- Dans un second temps il faut procéder à l'installation du binaire (Architecture AMD64).

```
cd /tmp
wget
https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
```

- Ensuite on extrait l'archive, on place l'exécutable dans le répertoire standard et on lui attribue les bons droits pour l'exécution.

```
tar xvfz node_exporter-1.8.2.linux-amd64.tar.gz
sudo mv node_exporter-1.8.2.linux-amd64/node_exporter /usr/local/bin/
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
```

- On nettoie les fichiers temporaires car des fichiers de script peuvent être mal exécutés ou utilisés à des fins malveillants.

```
rm -rf node_exporter-1.8.2.linux-amd64*
```

Maintenant place à la configuration de node exporter VPS:

- Cette fois-ci la configuration sera entièrement local donc l'adresse loop back 127.0.0.1 suffira.

- il faut créer le fichier de configuration node exporter dans le chemin suivant.

```
sudo nano /etc/systemd/system/node_exporter.service
```

- Copier dans le fichier cette configuration en y modifiant (`--web.listen-address`) avec l'IP loop back.

```
[Unit]
Description=Prometheus Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter \
    --web.listen-address=127.0.0.1:9100
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Activation du service node exporter:

- On recharge le daemon du systemd pour qu'il prenne en compte la nouvelle config, on fait en sorte que le service démarre au démarrage du Raspberry et on démarre le service.

```
sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
```

- Important: Voir si le service a bien démarré.

```
sudo systemctl status node_exporter
```

Tester ensuite node exporter:

- Vérifier que le port :9100 est bien ouvert et écoute sur la bonne IP

```
curl http://10.99.0.2:9100/metrics
```

Installer pour finir de la même façon que le premier VPS sur le deuxième VPS, aucune différence sauf l'adresse qui devient 10.99.0.3.

Mise en place de PostgreSQL en Docker Compose

Pour répondre au besoin du pôle concerné par la base de données, une instance PostgreSQL a été déployée sur le VPS via Docker Compose.

L'arborescence de travail a été créée avec :

```
sudo mkdir -p /opt/docker/database/postgres/data
sudo mkdir -p /opt/docker/database/postgres/init
cd /opt/docker/database
```

Le fichier .env contenant les identifiants a ensuite été créé :

```
sudo nano /opt/docker/database/.env
```

Contenu :

```
POSTGRES_DB=database
POSTGRES_USER=ynough
POSTGRES_PASSWORD=mot-de-passe-changé
```

Puis sécurisé :

```
sudo chmod 600 /opt/docker/database/.env
```

Un script d'initialisation SQL a aussi été ajouté :

```
sudo nano /opt/docker/database/postgres/init/01-init.sql
```

Contenu :

```
SELECT 'PostgreSQL init script loaded' AS info;
```

Le fichier Docker Compose a ensuite été préparé :

```
sudo nano /opt/docker/database/compose.yaml
```

Contenu :

```
services:
  postgres:
    image: postgres:17
    container_name: postgres
    env_file:
      - .env
    ports:
      - "5432:5432"
    volumes:
      - ./postgres/data:/var/lib/postgresql/data
      - ./postgres/init:/docker-entrypoint-initdb.d:ro
    restart: unless-stopped
```

Les permissions ont été appliquées sur les données :

```
cd /opt/docker/database
sudo chown -R 999:999 ./postgres/data
sudo chmod -R 700 ./postgres/data
```

Le conteneur a ensuite été lancé :

```
cd /opt/docker/database
sudo docker compose pull
sudo docker compose up -d
```

Enfin, le port PostgreSQL a été autorisé dans UFW :

```
sudo ufw allow 5432/tcp
```

À ce stade, la base de données était accessible à distance pour l'équipe.

Mise en place de Redis sur le VPS

Un service Redis a également été déployé sur le VPS afin de pouvoir servir de cache pour l'API de l'application.

Redis a été retenu car il s'agit d'une base clé/valeur en mémoire, adaptée aux cas d'usage suivants :
- cache
- stockage de sessions
- données temporaires
- file légère
- stockage intermédiaire entre services

L’arborescence a été créée ainsi :

```
sudo mkdir -p /opt/docker/redis/data
sudo mkdir -p /opt/docker/redis/config
cd /opt/docker/redis
```

Le fichier .env a ensuite été créé :

```
sudo nano /opt/docker/redis/.env
```

Contenu :

```
REDIS_PASSWORD=mot-de-passe-redis
```

Puis sécurisé :

```
sudo chmod 600 /opt/docker/redis/.env
```

Le fichier de configuration Redis a ensuite été préparé :

```
sudo nano /opt/docker/redis/config/redis.conf
```

Contenu :

```
bind 0.0.0.0
port 6379
appendonly yes
save 60 1000
protected-mode yes
requirepass mot-de-passe-redis
dir /data
```

Le réseau Docker backend a été créé avec :

```
sudo docker network create backend 2>/dev/null || true
```

Le fichier Docker Compose a ensuite été ajouté :

```
sudo nano /opt/docker/redis/compose.yaml
```

Contenu :

```
services:
  redis:
    image: redis:7.4-alpine
    container_name: redis
    command: ["redis-server", "/usr/local/etc/redis/redis.conf"]
    volumes:
      - ./data:/data
      - ./config/redis.conf:/usr/local/etc/redis/redis.conf:ro
```

```
    networks:
      - backend
    restart: unless-stopped

networks:
  backend:
    external: true
    name: backend
```

Les permissions sur les données ont ensuite été appliquées :

```
cd /opt/docker/redis
sudo chown -R 999:999 ./data
sudo chmod -R 700 ./data
```

Enfin, Redis a été lancé :

```
cd /opt/docker/redis
sudo docker compose pull
sudo docker compose up -d
```

Redis est alors prêt à être consommé par l'API.

Installer le tunnel VPN entre les deux VPS.

Installation:

```
sudo apt update
sudo apt install -y wireguard
```

Génération de clées:

```
sudo install -m 700 -d /etc/wireguard
cd /etc/wireguard

umask 077
wg genkey | sudo tee privatekey | wg pubkey | sudo tee publickey
```

Voir les clés et les noter (et mettre la clée publique sur le VPS…):

```
sudo cat /etc/wireguard/privatekey
sudo cat /etc/wireguard/publickey
```

Config:

```
sudo nano /etc/wireguard/wg0.conf
```

Contenu:

```
[Interface]
Address = 10.99.0.3/24
PrivateKey = CLE_PRIVEE_VPS2

[Peer]
PublicKey = CLE_PUBLIQUE_VPS1
Endpoint = 94.125.160.250:51820
AllowedIPs = 10.99.0.1/32
PersistentKeepalive = 25
```

Démarrer WireGuard sur le VPS2

```
sudo systemctl enable --now wg-quick@wg0
sudo systemctl status wg-quick@wg0
sudo wg show
```

Vérifications :

VPS2:

```
ping -c 4 10.99.0.1
sudo wg show
```

Depuis le VPS:

```
ping -c 4 10.99.0.3
sudo wg show
```

Si tout est bon, wg show affiche :

- un latest handshake
- du trafic transfer
- le peer correspondant

Supervision et gestion future

La supervision initiale mise en place repose principalement sur Prometheus et Grafana. Prometheus permet la collecte régulière de métriques, tandis que Grafana permet de les visualiser. Cette base pourra ensuite être enrichie avec davantage de cibles à superviser, plus de dashboards, et éventuellement des alertes.

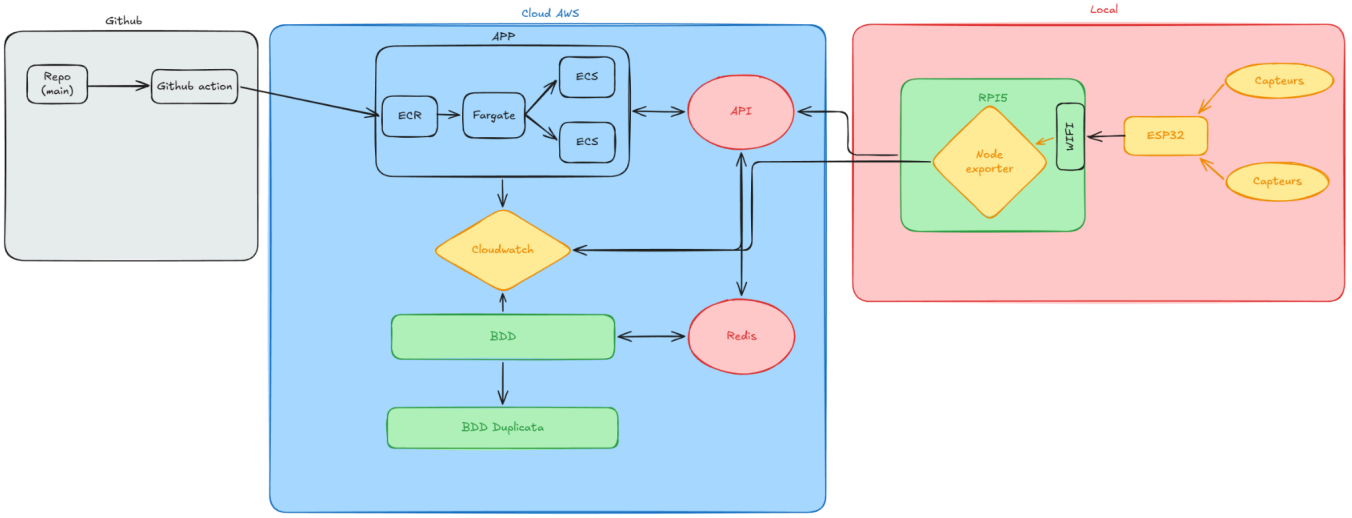
Concernant la gestion future de l'infrastructure, plusieurs axes d'amélioration sont déjà identifiés :
- mettre en place la réplication de la base de donnée et du code soruce
- mettre en place Redis sur l'API
- mieux restreindre les accès aux services exposés

- réduire l’exposition publique directe de PostgreSQL
- intégrer les futurs services applicatifs dans les réseaux Docker déjà préparés
- renforcer encore la documentation pour faciliter la maintenance et la reprise du projet

Proposition Cloud

Dans une vision future, on peut penser à l'expansion de notre application. En effet, notre infrastructure cloud actuelle n'est pas scalable : si le nombre d'utilisateurs explose ou si le VPS ne fonctionne plus, notre infrastructure est cassée(single point of failure). Pour pallier ce problème, on a décidé de pousser notre infrastructure en Cloud (AWS). Cela permet une scalabilité et une haute disponibilité à nos EC2 d’être situés à différents endroits dans le monde et d'avoir une EC2 située à Paris et une à Londres. Avec cette solution, on a une infrastructure multi-régions, on est donc protégé contre les pertes de données et l’indisponibilité.

Schéma de la proposition Cloud



Voici notre schéma pour le Cloud, il est réparti en 3 grandes parties : Local, Github, Cloud :

Local:

La partie locale va être la récupération des informations réelles de la partie et de les pousser vers le cloud.

Github:

Le GitHub est l’endroit où nous avons notre application mobile et web.Cette application web, nous pouvons l'héberger sur le cloud. Pour ce faire, l’application passe par GitHub Action Pipelines CI/CD permettant de passer notre application en image docker et de pouvoir la pousser plus aisément sur le cloud.

Cloud:

Pour cette partie, division la en plusieurs parties car elle est conséquente : APP, API, BDD, Redis et le CloudWatch

APP:

L'hébergement de notre application passe par 3 points importants : ECR, Fargate et ECS

ECR (Elastic Container Registry) permet le stockage de notre image et d'avoir une bibliothèque d'images. On ne stocke qu'une image pour l'instant mais cela permet la scalabilité d'autres applications futures.

Fargate le moteur de calcul "Serverless" pour les conteneurs, il donne la puissance à l'ECS et le nombre d'ECS à déployer

ECS, il récupère dans la bibliothèque ECR l'image et demande à Fargate la puissance nécessaire pour lancer le conteneur

API:

Pour réduire les coûts de l'API, nous avons pour but d'utiliser Flask + AWS Lambda + API Gateway. Cela nous permet d'avoir du serverless et d'être facturée à la requête et non au serveur.

BDD (Amazon RDS):

Pour la base de données, on utilise une instance RDS en PostgreSQL.

Redis:

On utilise Redis comme système de mise en cache pour la rapidité des réponses vers l'API.

Cloudwatch:

Pour la surveillance de nos logs / consommations ressources, on utilise AWS Cloudwatch qui va être le Grafana des machines dans AWS.

Le schéma du cloud expliqué, passons au déploiement.
Pour ce projet, nous avons décidé d'utiliser Terraform.

Terraform:

Terraform est un outil permettant de faire de l'IAC, donc de déployer des instances…etc directement avec des lignes de code.

Nous avons déployé les solutions ci-dessus :

Avant tout commencement, l'on donne à Terraform le providers et sa version minimale.

```
terraform {  Oiha_dev
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.92"
    }
  }

  required_version = ">= 1.2"
}
```

```
provider "aws" {  Oiha_dev
  region = "eu-west-3"
}

data "aws_vpc" "default" {  1 usage  Oiha_dev
  default = true
}

data "aws_subnets" "default" {  1 usage  Oiha_dev
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_iam_role" "ecs_execution" {  1 usage  Oiha_dev
  name = "ecsTaskExecutionRole"
}
```

Dans l'exécution du script `Provider"aws"` permet de dire à terraform que l'on est sur du AWS et de spécifier la régions dans laquelle on déploie nos services.

`aws_vpc` **:** Récupère l'identifiant du réseau créé automatiquement par AWS.

aws_subnets : Liste tous les sous-réseaux rattachés à ce VPC.
aws_iam_role : Récupère un rôle de sécurité existant (ecsTaskExecutionRole). ça permet au ECS d'aller chercher L'image dans L'ECR.

```
resource "aws_ecr_repository" "app" {  2 usages  Oiha_dev
  name = "ecr-repository"
}

output "ecr_url" {  Oiha_dev
  value = aws_ecr_repository.app.repository_url
}

resource "aws_ecs_cluster" "main" {  1 usage  Oiha_dev
  name = "ecs-cluster"
}
```

On crée un ECR et un clusters de ECS avec les parties **resource** et **erc_url** affiche L'url du dépôt pour la configuration de la Pipeline.

```
resource "aws_ecs_task_definition" "app" {  1 usage  Oiha_dev +1
  family                   = "ecs-task"
  requires_compatibilities = ["FARGATE"]
  network_mode             = "awsvpc"
  cpu                      = 256
  memory                   = 512
  execution_role_arn       = data.aws_iam_role.ecs_execution.arn

  container_definitions = jsonencode([{
    name         = "ecs-container"
    image        = "${aws_ecr_repository.app.repository_url}:latest"
    essential    = true
    portMappings = [{ containerPort = 80 }]
  }])
}
```

Définit la taille du serveur virtuel avec **cpu, memory.** On précise d'utiliser fargate et qu'on ouvre le port 80.

```
resource "aws_ecs_service" "app" {  no usages  Oiha_dev +1
  name            = "ecs-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  launch_type     = "FARGATE"
  desired_count   = 1

  network_configuration {
    subnets          = data.aws_subnets.default.ids
    assign_public_ip = true
  }
}
```

Ici on s'assure de la continuité du services avec `desired_count = 1` disant que l'on veut toujours une application qui tourne.

```
resource "aws_db_instance" "example" {  no usages  new *
  allocated_storage    = 20
  engine               = "postgres"
  engine_version       = "13"
  instance_class       = "db.t3.micro"
  db_name               = "mydb"
  username             = "testuser"
  password             = "votre_mot_de_passe_secret"
  skip_final_snapshot  = true
}
```

Création de l'instance de la DB en postegre et ces informations .
`instance_class = db.t3.micro` Choix des composant de la db ici est un exemple avec un free tiers.

Conclusion

Au final concernant le pôle infrastructure nous avons pu mettre en place les trois serveurs, le réseau sécurisé global (vpn, ufw et fail2ban) est déployé et fonctionnel. Le monitoring via la remontée des métriques de l'ensemble des serveurs vers le dashboard grafana est également opérationnel. Sur le VPS nous avons pu déployer la base de données PostgreSQL ainsi que le cache Redis, et pour la partie locale avec le raspberry nous avons

déployé la double connexion WiFi pour l’accès WAN du système ainsi que le hotspot local auquel nous avons déjà tenté de connecter les ESP avec succès.